ARTIFICIAL INTELLIGENCE

THANUSHIYA MAHESHWARAN

PART A: CLASSIFICATION USING MACHINE LEARNING

My report is based on the machine learning algorithms, I have been given two algorithms that needs to be followed by certain steps as asked in the assessment. In this the method and techniques used and the explanation report on given algorithm and data.

Given algorithms are: Logistic Regression and Decision Tree

PRE – PROCESSING OF THE DATA

```
In [1]: %matplotlib inline
        import numpy as np
        import matplotlib.pyplot as plt
        import pandas as pd
```

```
In [15]: dataset = pd.read_csv("C:\\Users\Admin\OneDrive\Desktop\Assignment2_data_files\diagnose4.csv")
```

```
In [16]: dataset.head()
```

Pre – processing data means a step that involve in transforming the raw data into a suitable format in which the training model requires in machine learning. This data pre-processing helps in addressing various data related issues of a model. There are some of the common techniques that involves in this data pre-processing method such as: data cleaning, data integration, data transformation, feature selection, feature encoding, and feature scaling and dimensionality reduction.

FEATURE SELECTION

```
In [16]: dataset.head()
```

Out[16]:

	ID	Gender	Location	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	
0	0	Male	Dublin	119.781071	249.975623	12.213626	616.747483	-67.801441	2245.295350	-173.357067	NaN	247.422864	651.401330	-527.9
1	1	Female	London	19.530750	134.721632	-6.651759	-1047.258296	403.349360	NaN	-56.120561	323.411436	2694.009119	766.443680	-313.1
2	2	Female	NaN	21.260438	-532.933756	4.203863	-899.699109	-325.946321	964.039710	35.871435	873.701879	1158.964157	877.331952	-386.1
3	3	Female	London	89.687882	-919.708815	0.274286	-2944.513415	175.185393	-753.952723	74.910322	328.956405	1363.735160	207.146672	
4	4	NaN	Belfast	179.957287	717.183364	-12.874797	-3960.304794	-286.928998	-1888.159139	61.087417	424.554386	3898.100368	228.118640	-3.9

x10	x11	Diagnosis
651.401330	-527.925560	0
766.443680	-313.190050	2
877.331952	-386.170945	0
207.146672	NaN	2
228.118640	-3.960441	2

In [20]:
```
X = dataset.iloc[:, 3:14].values
y = dataset.iloc[:, 14].values
d = dataset.iloc[:, 1:3].values #variables to print the values
```

In [21]:
```
print(dataset, "\n") #displays the dataset
```

```
      ID  Gender Location          x1           x2         x3           x4  \
0      0    Male   Dublin  119.781071   249.975623  12.213626   616.747483
1      1  Female   London   19.530750   134.721632  -6.651759 -1047.258296
2      2  Female      NaN   21.260438  -532.933756   4.203863  -899.699109
3      3  Female   London   89.687882  -919.708815   0.274286 -2944.513415
4      4     NaN  Belfast  179.957287   717.183364 -12.874797 -3960.304794
..   ...     ...      ...         ...          ...        ...          ...
382  382  Female   Dublin -194.055391  -500.652429  15.999716 -1468.684650
383  383    Male   London -270.626396   919.896384   6.712769 -1847.339710
384  384  Female   London  -11.262874 -2173.372192   5.798158 -2925.305358
385  385    Male   Dublin  -89.119059   -60.531635  12.174778  2259.782220
386  386    Male  Belfast  -84.978968  1098.339671  24.032667 -6023.576535

            x5           x6          x7          x8           x9  \
0    -67.801441  2245.295350 -173.357067         NaN   247.422864
1    403.349360          NaN  -56.120561  323.411436  2694.009119
2   -325.946321   964.039710   35.871435  873.701879  1158.964157
3    175.185393  -753.952723   74.910322  328.956405  1363.735160
4   -286.928998 -1888.159139   61.087417  424.554386  3898.100368
..          ...          ...         ...         ...          ...
382  -57.355126  1203.019068  378.095148 -798.540093  2212.617287
383  133.201595 -1178.736989  103.235299  102.494970   632.083495
384 -459.414527 -1290.160664   24.286726  424.701100   874.014093
385  249.382348  6113.692714  -84.651547 -236.258652  1296.626212
386  240.205391   831.390067 -283.810523 -474.436469  2459.947035

             x10         x11  Diagnosis
0     651.401330 -527.925560          0
1     766.443680 -313.190050          2
2     877.331952 -386.170945          0
3     207.146672         NaN          2
4     228.118640   -3.960441          2
..           ...         ...        ...
382   213.316224  148.577793          1
383  -469.514914 -250.137753          2
384  -291.380199 -387.057984          2
385   469.936538 -183.924858          0
386  1265.657245  -41.543536          0

[387 rows x 15 columns]
```

In [22]:
```
print(X, "\n")
```

```
[[  119.7810709    249.9756233     12.21362552 ...   247.4228637
    651.4013304   -527.9255605 ]
 [   19.53075024   134.7216323     -6.65175862 ...  2694.009119
    766.4436796   -313.1900497 ]
 [   21.26043788  -532.9337564      4.20386309 ...  1158.964157
    877.331952    -386.1709448 ]
 ...
 [  -11.26287355 -2173.372192       5.7981584  ...   874.0140932
   -291.380199    -387.0579836 ]
 [  -89.11905889   -60.53163459    12.17477759 ...  1296.626212
    469.936538    -183.9248585 ]
 [  -84.97896776  1098.339671      24.03266651 ...  2459.947035
   1265.657245     -41.54353616]]
```

```
In [23]: print(y) #prints the numerical values
```

```
[0 2 0 2 2 2 2 1 2 2 1 0 2 1 2 2 1 1 1 2 2 2 2 1 1 0 1 0 1 2 2 2 0 0 1 0 2
 1 0 1 0 1 0 2 0 2 0 0 0 1 0 1 2 1 0 1 0 1 2 0 0 1 0 0 1 0 1 2 2 1 2 1 1 2
 1 0 2 2 1 1 0 2 2 0 1 2 2 1 1 1 0 2 0 2 0 0 1 2 1 1 2 0 0 0 2 0 0 1 2 0 0
 2 1 1 1 1 0 2 2 0 0 2 1 1 2 1 2 2 0 0 0 1 1 2 0 2 2 0 0 2 0 0 1 0 2 2 0 0
 0 2 0 2 0 2 0 0 0 1 0 0 0 0 1 1 2 0 1 0 2 1 0 1 0 2 0 0 0 1 0 0 2 2 1 0 0
 0 2 0 2 0 1 2 2 0 2 1 1 2 1 2 1 1 0 2 2 0 0 0 0 2 0 0 1 2 1 1 2 2 1 2 2 2
 2 2 1 0 1 0 1 1 0 0 0 2 2 2 1 2 1 0 0 0 1 2 1 1 2 2 0 1 0 2 1 2 2 2 1 0 1
 0 1 2 1 2 1 1 0 1 0 1 2 1 0 0 2 1 1 0 2 0 0 1 2 2 2 2 0 2 0 1 2 1 0 1 2 1
 1 0 2 0 2 1 1 0 0 1 2 0 1 1 0 1 1 1 0 1 2 1 2 2 1 0 2 0 2 0 1 2 1 1 2 0 0
 1 1 2 2 2 2 2 1 1 0 0 2 2 2 1 0 1 2 1 1 2 0 1 0 2 0 1 2 1 2 1 1 0 1 0 2 1
 0 1 1 1 2 0 0 1 1 1 2 1 1 2 2 0 0]
```

```
In [24]: print(d) #to print dataset as presented
```

```
[['Male' 'Dublin']
 ['Female' 'London']
['Female' nan]
 ['Female' 'London']
 [nan 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
```

```
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'Belfast'] ['Female' 'London']
 ['Female' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
```

```
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Belfast'] ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Female' 'Belfast'] ['Male'
 'Dublin']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
```

```
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'Dublin']
 ['Male' 'Belfast'] ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
```

```
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London'] ['Female' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'Dublin']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
```

```
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London'] ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'Belfast']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
['Male' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'Belfast']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
```

```
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast'] ['Male'
 'London']
 ['Female' 'Belfast']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London'] ['Female' 'Dublin']
 ['Female' 'Dublin'] ['Male'
 'Belfast']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Male' 'Belfast']]
```

```
In [25]: display(dataset.describe()) #gives the summary of the data set
```

	ID	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	
count	387.000000	386.000000	387.000000	386.000000	387.000000	387.000000	385.000000	387.000000	386.000000	386.000000	387.000000	386.
mean	193.000000	4.558917	-135.075281	1.691165	-965.123825	6.023456	-11.073828	-12.998791	5.436705	617.733268	273.478284	15.
std	111.861522	143.244603	941.910447	11.442658	2094.133191	303.625270	1752.957266	148.978073	583.204343	1439.397343	650.605008	565.
min	0.000000	-408.555430	-2527.106474	-29.255353	-8579.708089	-862.038082	-5364.743520	-481.411085	-1537.879728	-3670.090485	-1920.603675	-1408.
25%	96.500000	-88.812203	-886.712296	-5.588551	-2128.884273	-176.398292	-1236.870172	-117.853095	-385.835199	-419.534368	-89.718706	-380.
50%	193.000000	11.064327	-36.085223	2.633369	-950.204162	21.704146	-195.986971	-6.600129	13.409822	752.490560	357.336728	54.
75%	289.500000	103.796505	554.705084	9.570294	319.288943	211.762825	1167.590425	79.842394	424.098472	1583.655809	700.359853	402.
max	386.000000	430.277110	2239.062538	40.891061	5589.189837	852.725612	6113.692714	508.614533	1324.854926	4264.831638	1820.457605	1713.

x11	Diagnosis
386.000000	387.000000
15.774216	0.994832
565.881641	0.817537
-1408.558100	0.000000
-380.991264	0.000000
54.229708	1.000000
402.302187	2.000000
1713.494778	2.000000

IDENTIFICATION OF MISSING VALUES

In the Data pre-process method there will be issue that will cause errors in the algorithms. Therefore I have used the **Simple Imputer** from the **Scikit-learn** in order to find out the missing values in the data pre-process technique. In here the missing values will be shown in two different ways in Numerical data and textual data.

```
In [27]: from sklearn.impute import SimpleImputer
         imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
         X[:,0:13] = imputer.fit_transform(X[:,0:13])
         print(X) #printing this to check whether every columns holds a value
```

```
[[  119.7810709    249.9756233     12.21362552 ...   247.4228637
    651.4013304   -527.9255605 ]
 [   19.53075024   134.7216323     -6.65175862 ...  2694.009119
    766.4436796   -313.1900497 ]
 [   21.26043788  -532.9337564      4.20386309 ...  1158.964157
    877.331952    -386.1709448 ]
 ...
 [  -11.26287355 -2173.372192       5.7981584  ...   874.0140932
   -291.380199    -387.0579836 ]
 [  -89.11905889   -60.53163459    12.17477759 ...  1296.626212
    469.936538    -183.9248585 ]
 [  -84.97896776  1098.339671      24.03266651 ...  2459.947035
   1265.657245     -41.54353616]]
```

The below mentioned images shows the code that is given for the output of textual data that are in the columns of the given dataset. This provides a list of text / data that is from the model.

```
In [28]: imputer = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
         d = imputer.fit_transform(d)
         print(d) #to print the first and second columns to see if it works successfully
```

```
[['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
```

```
 ['Female' 'Belfast']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
```

```
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'Dublin'] ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
```

```
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London'] ['Female' 'London'] ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'Dublin']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
```

```
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
 ['Female' 'Dublin']
['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'Dublin'] ['Female' 'Dublin']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Male' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'Dublin']
['Male' 'Dublin']
 ['Female' 'London']
['Male' 'Belfast']
```

```
 ['Female' 'Dublin']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'Belfast']
 ['Female' 'London'] ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'Dublin']
 ['Male' 'Belfast']
['Male' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
```

```
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Belfast']
 ['Female' 'Belfast']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Female' 'Belfast']
 ['Female' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'Dublin']
 ['Female' 'Dublin'] ['Male'
 'Belfast']
 ['Female' 'London'] ['Male'
 'London']
 ['Female' 'London']
 ['Male' 'London'] ['Female' 'London'] ['Male' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'London'] ['Male'
 'Dublin']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'London']
 ['Male' 'London']
 ['Male' 'London']
 ['Male' 'Dublin']
 ['Male' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'London']
 ['Male' 'London']
```

```
 ['Male' 'Belfast']
 ['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
['Male' 'Belfast']
 ['Female' 'London']
 ['Female' 'London']
 ['Male' 'Belfast']
 ['Male' 'London']
 ['Female' 'Dublin']
['Male' 'London']
 ['Female' 'London']
['Male' 'Dublin']
 ['Male' 'Belfast']]
```

CONVERSION OF DATA

```
In [30]: from sklearn.preprocessing import OneHotEncoder
         encoder = OneHotEncoder()
         # to process the number of elements needed in the columns

         # d2 identifies the first column of d
         d2 = d[:,0].reshape(-1,1)

         # OneHotEncoder is to fit the d2 column in order to transfer the data
         d3 = encoder.fit_transform(d2).toarray()

         # to combine the new features in column d
         d = np.concatenate((d3,d[:,1:2]),axis=1)

         d=d[:, 0:]
         print(d)

         [[0.0 1.0 'Dublin']
          [1.0 0.0 'London']
          [1.0 0.0 'London']
          ...
          [1.0 0.0 'London']
          [0.0 1.0 'Dublin']
          [0.0 1.0 'Belfast']]
```

FEATURE SCALING

This is a technique that is used in the data pre-processing step in order to standardize or normalize the feature of a dataset. And also this transforms the numerical values to a specific range or distribution of a dataset.

In [31]:
```
from sklearn.preprocessing import StandardScaler #this is used to scale the datasets

sc = StandardScaler() #Creating a StandardScaler

X[:,0:13] = sc.fit_transform(X[:,0:13]) #Scaling the features of X

print(X[1:, :]) #to print the result
```

```
[[ 0.10479048  0.28680663 -0.73099847 ...  1.44620372  0.75868389
  -0.58283851]
 [ 0.11689687 -0.42294202  0.22016001 ...  0.37698754  0.92934322
  -0.71214154]
 [ 0.59583252 -0.83410184 -0.12414554 ...  0.5196182  -0.10208572
   0.        ]
 ...
 [-0.11073948 -2.16680417  0.35985053 ...  0.17850917 -0.86932883
  -0.71371314]
 [-0.65566854  0.07924335  0.91856339 ...  0.47287431  0.3023533
  -0.35381445]
 [-0.62669132  1.31117731  1.95753949 ...  1.28317073  1.52698384
  -0.10155204]]
```

Configuration of mean value which should be 0 after the feature scaling.

In [33]:
```
print('Mean of x1: {:5.3f}\n'.format(np.mean(X[:,0])))
print('Mean of x2: {:5.3f}\n'.format(np.mean(X[:,1])))
print('Mean of x3: {:5.3f}\n'.format(np.mean(X[:,2])))
print('Mean of x4: {:5.3f}\n'.format(np.mean(X[:,3])))
print('Mean of x5: {:5.3f}\n'.format(np.mean(X[:,4])))
print('Mean of x6: {:5.3f}\n'.format(np.mean(X[:,5])))
print('Mean of x7: {:5.3f}\n'.format(np.mean(X[:,6])))
print('Mean of x8: {:5.3f}\n'.format(np.mean(X[:,7])))
print('Mean of x9: {:5.3f}\n'.format(np.mean(X[:,8])))
print('Mean of x10: {:5.3f}\n'.format(np.mean(X[:,9])))
print('Mean of x11: {:5.3f}\n'.format(np.mean(X[:,10])))
print (X)
```

```
Mean of x1: 0.000

Mean of x2: 0.000

Mean of x3: -0.000

Mean of x4: -0.000

Mean of x5: -0.000

Mean of x6: -0.000

Mean of x7: 0.000

Mean of x8: 0.000

Mean of x9: -0.000

Mean of x10: -0.000

Mean of x11: -0.000
```

```
[[ 0.80646002  0.40932697  0.9219672  ... -0.25793503  0.58163135
  -0.96329358]
 [ 0.10479048  0.28680663 -0.73099847 ...  1.44620372  0.75868389
  -0.58283851]
 [ 0.11689687 -0.42294202  0.22016001 ...  0.37698754  0.92934322
  -0.71214154]
 ...
 [-0.11073948 -2.16680417  0.35985053 ...  0.17850917 -0.86932883
  -0.71371314]
 [-0.65566854  0.07924335  0.91856339 ...  0.47287431  0.3023533
  -0.35381445]
 [-0.62669132  1.31117731  1.95753949 ...  1.28317073  1.52698384
  -0.10155204]]
```

<u>FEATURE SELECTION (“k” feature)</u>

Feature selection is a step that is taken to achieve the goal in selecting a subset from the given dataset to improve the performance and interpretability. This process involves in several approaches as feature selection. Such as: Filter methods, Wrapper methods, embedded methods and Dimensionality reduction.

```
In [34]: from sklearn.feature_selection import SelectKBest
         from sklearn.feature_selection import mutual_info_regression

         select = SelectKBest(mutual_info_regression, k=11).fit(X, y) #Identify the features neeeded which fits for k

         X = select.transform(X) #Transform the features of X

         print(X) #Print the features of X
```

```
[[ 0.80646002  0.40932697  0.9219672  ... -0.25793503  0.58163135
  -0.96329358]
 [ 0.10479048  0.28680663 -0.73099847 ...  1.44620372  0.75868389
  -0.58283851]
 [ 0.11689687 -0.42294202  0.22016001 ...  0.37698754  0.92934322
  -0.71214154]
 ...
 [-0.11073948 -2.16680417  0.35985053 ...  0.17850917 -0.86932883
  -0.71371314]
 [-0.65566854  0.07924335  0.91856339 ...  0.47287431  0.3023533
  -0.35381445]
 [-0.62669132  1.31117731  1.95753949 ...  1.28317073  1.52698384
  -0.10155204]]
```

SPLITTING OF DATA

Data splitting is an essential method that involves in dividing the dataset into multiple subsets of data. This evaluates the machine learning model of a dataset. There are some of the commonly used data splitting, such as: Train test split, Train validation test split, K-fold cross validation, Stratified splitting.

```
In [37]: from sklearn.model_selection import train_test_split

         X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.1,random_state = 0) #test the data

         #to print the test data of x and y
         print(X_test)
         print(y_test)
```

```
[[-3.99774398e-01  7.67748317e-01 -1.62785931e+00  1.82685176e+00
   1.57006964e+00 -2.03829877e-01 -3.90123650e-01  2.26822872e+00   -
1.15513053e+00  5.51189033e-01 -4.09377586e-01]

 [-2.07334273e+00  9.79881916e-01  5.19026028e-01  6.53901281e-01
  -1.22516054e+00 -4.52410151e-01  9.54158887e-01 -1.06548721e+00   -
2.00905105e-01  1.20392745e+00  3.94202171e-02]

 [-1.06631143e+00  1.10998780e+00  2.70156105e-01 -7.16741237e-02
   9.16375581e-01  1.78426155e-01 -1.15425966e+00 -8.00072805e-01   -
1.10941053e+00 -4.83165944e-01 -1.18248335e+00]

 [-3.50330245e-02 -5.69959930e-01  4.99713065e-01 -8.37245120e-01
  -3.98930784e-01  6.75007586e-01 -2.66334926e-01  2.48327451e-01
5.34865722e-01  1.76000182e+00 -1.01032399e+00]

 [-1.29640339e+00 -2.27906406e-01 -9.77544953e-01  7.46854361e-01
   7.55872917e-01 -1.55464274e+00 -2.91113032e-01  1.09733924e+00   -
1.28751794e+00  1.38779001e+00 -8.03500191e-01]

 [-6.35391764e-01 -8.42432974e-04  8.29957674e-01  4.70308274e-01
   1.02218728e+00  1.10494119e+00 -3.96432594e-01 -1.79736453e+00   -
1.04139364e-01 -4.21718508e-01 -7.44255042e-01]

 [-2.29404403e+00  4.37481975e-01 -2.91299147e-01  8.98712297e-01
   1.77520567e-01 -6.37086569e-01  1.07561884e+00 -3.20158172e-01   -
1.59179486e+00  9.23766590e-01  7.64495247e-01]

 [ 1.35296752e+00 -5.60214258e-01 -4.72079525e-01 -2.79181257e-01
  -1.77279236e-01 -7.01999598e-01  5.60833784e-01 -5.25167188e-01
8.91177342e-01 -3.61022397e-01 -1.91090848e+00]

 [ 7.57504562e-01  7.59981261e-01  1.42821395e-01  4.19075320e-01
   1.21425748e+00 -5.00813796e-01 -9.54278312e-01  8.05802893e-02
2.35435197e-01  1.15731540e+00 -3.39225923e-01]

 [ 1.14715228e+00 -5.83627792e-01  7.86714621e-01 -2.82621966e+00
  -1.35769486e+00  4.76379041e-01  4.73005357e-01  1.20598155e+00
1.32095001e+00  1.14377164e+00  8.26901685e-01]

 [ 6.59496793e-01 -2.50221787e+00 -1.85408503e+00 -4.39123022e-02
   2.50582190e-02 -1.32344525e+00  4.35797157e-01 -2.50483118e-01
-7.06166304e-01 -1.52312928e+00  1.19769824e+00]
```

```
[ 2.20710424e+00  1.07666592e+00  1.59012976e-01 -5.58007195e-03
 -5.85433423e-01  1.35484542e+00 -7.78950404e-01 -1.72468138e+00
  4.40411800e-01 -3.04372896e+00 -7.59906013e-01]

[-5.00097955e-01 -1.22082313e+00 -4.00878066e-01  6.08162413e-01
  5.08583509e-01 -8.98793527e-02  5.33619810e-01 -2.17417125e-01
  6.60823463e-02  3.28347738e-01  3.69582452e-01]

[-6.83421973e-01  1.07641667e+00  3.99239325e-01  5.76227586e-01
  5.39787539e-01  1.03198835e+00  4.06967508e-01  1.23639459e-01
 -1.11365082e+00  6.67705570e-01  8.00482069e-01]

[-4.39554829e-01  1.47639354e+00  1.17330180e+00  2.42866011e-01
  1.97307817e-01  1.73548818e+00  5.66859923e-01  1.70947312e+00
  2.42645060e-01 -1.59670762e+00  1.46344496e-01]

[-7.07858077e-01 -9.95476142e-01  7.12592969e-02  7.22008898e-01
  3.05164484e-01  1.34301118e+00 -1.90008838e-01 -2.96618900e-01
 -1.31393331e+00 -1.53935213e+00 -4.01078596e-01]

[-1.62822084e-01 -1.77854626e+00  7.73067589e-01 -1.59650716e+00
  5.17117322e-02 -2.97524689e-01  6.86528702e-01 -2.40873184e-01
  8.69142965e-01 -1.30319341e+00 -1.97134026e+00]

[-1.07463006e-01  4.37905177e-01 -8.73432831e-01  3.38982647e-01
 -1.06133767e+00 -2.82829162e-01 -1.63037355e+00 -3.34821232e-01
  6.65093257e-02  1.88610774e+00  1.32419486e+00]

[-5.72911507e-01  1.17755474e+00  1.22570092e+00  9.41167349e-01
 -1.63933079e+00 -4.08784570e-01 -5.45754589e-01  1.79442048e+00
 -9.00335643e-01  2.25764974e-02  6.22318279e-01]

[-1.50470070e-01  1.01176372e+00 -7.07341709e-01 -9.63229577e-01
 -2.00106273e+00 -2.72616173e-01  1.93783716e+00 -6.03731452e-01
  1.00584354e+00  7.38680645e-01  5.31538319e-01]

[-1.39013629e+00 -3.88625467e-01  1.25370056e+00 -2.40773932e-01
 -2.09009696e-01  6.95297166e-01  2.62857616e+00 -1.38212679e+00
  1.11089632e+00 -9.25906499e-02  2.35293152e-01]

[ 1.49807896e+00  1.11520849e+00 -1.11815536e-01  1.28002315e+00
 -2.17885579e+00  7.56162465e-02  1.29717583e+00 -3.22553808e-01
 -1.93408058e+00  2.33121413e-01  4.58894552e-01]

[-2.20016990e+00  6.64877688e-01  2.00804871e+00  7.53297669e-03
  1.84431472e-01  1.30845836e+00  3.57546336e-01 -8.54966319e-01
 -1.43120175e+00 -5.79353513e-01 -1.86964913e-01]

[ 1.08213810e+00  7.14503225e-01  1.25232269e+00  2.45958520e-01
 -7.76269582e-01 -2.67287134e-01  3.03502796e-01 -1.27958060e+00
 -2.96322346e-01  1.86315592e-01  1.07959067e-01]

[ 1.08018915e+00 -2.54284012e+00  7.92569556e-01 -1.43673769e+00
 -4.83534610e-01 -9.91710147e-01  1.74039905e-01  1.67581844e-01
 -5.61406724e-03 -1.72752176e+00  7.69081829e-01]
```

```
 [-5.08008052e-01  3.74834950e-01  2.27291156e-01  1.71645866e-01
  -3.16646762e-01  2.87773543e-02 -1.36772973e-01 -4.44768350e-01
   4.47794651e-01   2.83857684e-01 -1.32245558e+00]
[ 4.45629042e-01 -1.07967213e+00  3.47343926e-01 -
7.63424607e-01
   7.96746505e-01  3.62115096e-01 -4.12856726e-01  3.09125661e-01
   5.19029249e-01 -3.95591434e-01 -2.53741164e-01]

 [ 8.55561756e-01  1.05170483e+00  2.58112882e-01 -5.42002810e-01
  -1.12863036e+00  4.11642389e-01  8.43285852e-01 -4.30133126e-01
  -6.77681843e-01 -5.53855313e-01  6.18036171e-01]

 [ 1.11436183e+00  8.51933605e-01  7.50448972e-01  1.96403508e-01
   1.32043962e+00 -5.19828024e-01  6.57261868e-01 -6.00300186e-01
  -4.17386198e-01  2.99366297e-01  9.64657793e-02]

 [ 1.07033316e+00  8.82145789e-01 -1.12023401e+00  8.52247701e-01
  -5.14533279e-01 -2.07355234e+00 -1.56512661e+00  9.53252353e-01
   2.51439855e-01 -4.67364885e-01  9.56355638e-01]

 [ 4.22494767e-01  9.99374501e-01  2.75777406e-01 -7.75279755e-02
   8.21377074e-01 -2.85388945e-01  3.41990525e-01  3.39830621e-01
   5.71120628e-01  8.04956637e-01 -1.08889949e-01]

 [ 3.90337801e-01  7.81222910e-02  1.32849396e+00 -5.51267959e-02
   1.22468910e+00 -4.71998264e-01  1.12195347e+00 -3.44653882e-01
  -4.74476467e-01  5.18429158e-01 -1.23250255e+00]

 [ 5.46813840e-01  3.06458937e-01 -1.14945141e+00  8.50634792e-01
  -9.91805488e-01 -1.35146673e+00 -1.26321883e+00  3.43415647e-01
  -5.47311578e-01  8.12726700e-01 -1.42022127e+00]

 [-1.95538121e-01  9.05648645e-01  3.56027965e-01  8.61056859e-01
  -1.60427599e+00 -1.07240956e+00  6.66622522e-01 -1.30569220e+00
  -1.13132425e+00  7.37370324e-01  1.10204602e-01]

 [ 6.57885468e-01 -1.40719678e+00  2.89099366e-01 -1.57979336e-01
  -5.23353841e-02 -8.38755472e-01 -4.67069872e-01  8.26236606e-02
  -7.96582824e-01 -1.00687265e+00 -1.56069655e-01]

 [ 1.04790483e-01  2.86806634e-01 -7.30998470e-01 -3.92719976e-02
   1.31030017e+00 -4.06919728e-18 -2.89825140e-01  5.46634423e-01
   1.44620372e+00  7.58683889e-01 -5.82838508e-01]

 [ 2.16898159e+00  9.20019846e-03 -1.40804641e+00  5.97772852e-01
   1.38886890e+00 -5.14227247e-01 -1.04490808e-01  1.34239748e+00
   3.56203850e-01  8.52746294e-01  7.20953383e-02]

 [ 3.76031844e-01 -1.41507081e+00 -5.53336912e-01 -6.83100585e-02
   5.12440491e-01 -1.24392123e-01 -1.51436238e+00  9.76358131e-01
   7.59727405e-01 -1.18728165e+00  3.68694814e-02]

 [-2.30790640e-01 -1.75226830e+00 -9.48239710e-01 -1.21622747e+00
   4.75010924e-01 -2.04126746e-01 -1.36226854e+00  2.02346197e-01
   6.42621897e-02 -3.14912087e-01 -1.86039639e-01]]

[0 2 1 0 1 1 1 0 2 0 1 1 1 1 1 0 2 2 2 2 1 1 1 2 2 0 0 1 2 1 2 2 1 0 2 2 2
```

```
1 0]
```

10-FOLD CROSS-VALIDATION

```
In [39]: from sklearn.naive_bayes import GaussianNB
         from sklearn.model_selection import KFold, cross_validate
         from sklearn.feature_selection import SelectKBest
         from sklearn.feature_selection import f_classif #this is to create a Classifier for the Naive Bayes

         nb = GaussianNB()

         cv = KFold(n_splits=10, shuffle=True, random_state = 0) # apply the k-Fold for the Cross Validation

         for kfs in range(1,11): #to identify the features of the training data

             select = SelectKBest(score_func=f_classif, k=kfs).fit(X_train, y_train)

             X_trainFS = select.transform(X_train)
             X_testFS = select.transform(X_test) #to test the inputs and transform
             scores = cross_validate(estimator = nb, X = X_trainFS, y = y_train, cv = cv)
             print(scores["test_score"].mean())

         0.5373109243697479
         0.6202521008403361
         0.6694117647058825
         0.6926050420168067
         0.6926050420168067
         0.6898319327731093
         0.695546218487395
         0.692436974789916
         0.6837815126050419
         0.6868067226890756
```

LOGISTIC REGRESSION

Logistic Regression is an algorithm in machine learning which is used as the binary classification. Logistic Regression works on several tasks such as: data preparation, feature scaling, model training, model optimization and prediction.

```
In [40]: from sklearn.linear_model import LogisticRegression #to import the Logistic Regression Classifier

         lr = LogisticRegression() #create an object

         lr.fit(X_trainFS, y_train) #to fit the model data sets into the training data

         y_pred = lr.predict(X_testFS) #predict the test data
```

CONFUSION MATRIX

Confusion matrix summarizes the performances of a classified table in which the model is presented by the predicted and actual classes of a dataset.

In [41]:
```
from sklearn.metrics import confusion_matrix #to import the confusion matrix

cm = confusion_matrix(y_test, y_pred) #to create the object and test the predicted target

print(cm) #to print the confusion matrix
```

```
[[4 2 3]
 [5 7 4]
 [2 3 9]]
```

PERFORMANCE METRICS

In [42]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score,f1_score
print('\nAccuracy: {:.2f}\n'.format(accuracy_score(y_test, y_pred)))
print('Micro Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='micro')))
print('Micro Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='micro')))
print('Micro F1-score: {:.2f}\n'.format(f1_score(y_test, y_pred,average='micro')))
print('Macro Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='macro')))
print('Macro Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='macro')))
print('Macro F1-score: {:.2f}\n'.format(f1_score(y_test, y_pred,average='macro')))
print('Weighted Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='weighted')))
print('Weighted Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='weighted')))
print('Weighted F1-score: {:.2f}'.format(f1_score(y_test, y_pred,average='weighted'))) #to print and test the accuracy
```

```
Accuracy: 0.51

Micro Precision: 0.51
Micro Recall: 0.51
Micro F1-score: 0.51

Macro Precision: 0.50
Macro Recall: 0.51
Macro F1-score: 0.50

Weighted Precision: 0.53
Weighted Recall: 0.51
Weighted F1-score: 0.51
```

Activate \
Go to Settin

TUNNING LOGISTIC REGRESSION MODEL

In [44]:
```
from sklearn.datasets import make_blobs
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

X, y = make_blobs(n_samples=1000, centers=2, n_features=13, cluster_std=20)

model = LogisticRegression() #to identify the models and the parameters of the data set
solvers = ['newton-cg', 'lbfgs', 'liblinear']
penalty = ['l2']
c_values = [100, 10, 1.0, 0.1, 0.01]

grid = dict(solver=solvers,penalty=penalty,C=c_values)
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0)
grid_search = GridSearchCV(estimator=model, param_grid=grid, n_jobs=-1, cv=cv,scoring='accuracy',error_score=0)
grid_result = grid_search.fit(X, y) #for the grid search and the results

print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_)) #to print the grid results
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

```
Best: 0.762333 using {'C': 0.01, 'penalty': 'l2', 'solver': 'liblinear'}
0.761667 (0.041640) with: {'C': 100, 'penalty': 'l2', 'solver': 'newton-cg'}
0.761667 (0.041640) with: {'C': 100, 'penalty': 'l2', 'solver': 'lbfgs'}
0.761667 (0.041640) with: {'C': 100, 'penalty': 'l2', 'solver': 'liblinear'}
0.761667 (0.041640) with: {'C': 10, 'penalty': 'l2', 'solver': 'newton-cg'}
0.761667 (0.041640) with: {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
0.761667 (0.041640) with: {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}
0.761667 (0.041640) with: {'C': 1.0, 'penalty': 'l2', 'solver': 'newton-cg'}
0.761667 (0.041640) with: {'C': 1.0, 'penalty': 'l2', 'solver': 'lbfgs'}
0.761667 (0.041640) with: {'C': 1.0, 'penalty': 'l2', 'solver': 'liblinear'}
0.761667 (0.041640) with: {'C': 0.1, 'penalty': 'l2', 'solver': 'newton-cg'}
0.761667 (0.041640) with: {'C': 0.1, 'penalty': 'l2', 'solver': 'lbfgs'}
0.761667 (0.041640) with: {'C': 0.1, 'penalty': 'l2', 'solver': 'liblinear'}
0.761667 (0.041640) with: {'C': 0.01, 'penalty': 'l2', 'solver': 'newton-cg'}
0.761667 (0.041640) with: {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
0.762333 (0.042007) with: {'C': 0.01, 'penalty': 'l2', 'solver': 'liblinear'}
```

RFE (RECURSIVE FEATURE ELIMINATION)

RFE is known as the Recursive Feature Elimination, this feature is commonly used to identify the most important variables for a given predicted task. Therefore Recursive Feature Elimination process typically works by choosing an estimator, feature ranking, feature elimination, model evaluation, stopping criterion and final model.

In [45]:
```
from sklearn.linear_model import LogisticRegression #Logistic Regression classification
from numpy import mean
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.pipeline import Pipeline
# define the dataset and create classification for the pipeline
X, y= make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

rfe = RFE(estimator=LogisticRegression(), n_features_to_select=13)
model = LogisticRegression(C= 100, penalty= 'l2', solver= 'newton-cg')
pipeline= Pipeline(steps=[('s',rfe),('m',model)]) # evaluation of the  model for Logistic Regression

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0)
n_scores = cross_val_score(pipeline, X, y, scoring='accuracy', cv=cv,n_jobs=-1, error_score='raise')

print('Accuracy score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #Print accuracy score
```

```
Accuracy score: 0.913 (0.027)
```

In [46]:
```
from sklearn.linear_model import LogisticRegression
from numpy import mean
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.pipeline import Pipeline # define the dataset and the classification for the RFE

X, y= make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)
rfe = RFE(estimator=LogisticRegression(), n_features_to_select=13)
model = LogisticRegression(C= 100, penalty= 'l2', solver= 'newton-cg')
pipeline= Pipeline(steps=[('s',rfe),('m',model)]) #pipeline created for the model for Logistic Regression

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0)
n_scores = cross_val_score(pipeline, X, y, scoring='precision', cv=cv,n_jobs=-1, error_score='raise')

# report performance
print('Precision score: %.3f (%.3f)' % (mean(n_scores), std(n_scores)))
```

```
Precision score: 0.930 (0.036)
```

In [47]:
```
from sklearn.linear_model import LogisticRegression #to estimate the  RFE for classification
from numpy import mean
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.pipeline import Pipeline

#to define the dataset
X, y= make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

rfe = RFE(estimator=LogisticRegression(), n_features_to_select=13)
model = LogisticRegression(C= 100, penalty= 'l2', solver= 'newton-cg')
pipeline= Pipeline(steps=[('s',rfe),('m',model)])

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0) #evaluation of the model
n_scores = cross_val_score(pipeline, X, y, scoring='recall', cv=cv, n_jobs=-1,error_score='raise')

print('Recall score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print and recall the score
```

```
Recall score: 0.894 (0.038)
```

In [48]:
```
from sklearn.linear_model import LogisticRegression #to estimate the  RFE for classification
from numpy import mean
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.pipeline import Pipeline
#to estimate the  RFE for classification
X, y= make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

rfe = RFE(estimator=LogisticRegression(), n_features_to_select=13)
model = LogisticRegression(C= 100, penalty= 'l2', solver= 'newton-cg')
pipeline= Pipeline(steps=[('s',rfe),('m',model)])

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0) #evaluation of the model
n_scores = cross_val_score(pipeline, X, y, scoring='f1', cv=cv, n_jobs=-1,error_score='raise')

print('F1-score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print and recall the score
```

```
F1-score: 0.911 (0.028)
```

DECISION TREE

This machine learning algorithm is used for both classification and regression tasks.

In [49]:
```
from sklearn.tree import DecisionTreeClassifier #this is for Decision Tree Classification

dt = DecisionTreeClassifier(criterion = 'entropy', random_state = 0, ccp_alpha= 0.0)
dt.fit(X_trainFS, y_train)

# to Predict the test cases of decision tree classification
y_pred = dt.predict(X_testFS)

# Construction of the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[ 3  3  3]
 [ 4 11  1]
 [ 1  0 13]]
```

In [50]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score,f1_score
print('\nAccuracy: {:.2f}\n'.format(accuracy_score(y_test, y_pred)))

print('Micro Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='micro')))
print('Micro Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='micro')))
print('Micro F1-score: {:.2f}\n'.format(f1_score(y_test, y_pred,average='micro')))

print('Macro Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='macro')))
print('Macro Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='macro')))
print('Macro F1-score: {:.2f}\n'.format(f1_score(y_test, y_pred,average='macro')))

print('Weighted Precision: {:.2f}'.format(precision_score(y_test, y_pred,average='weighted')))
print('Weighted Recall: {:.2f}'.format(recall_score(y_test, y_pred,average='weighted')))
print('Weighted F1-score: {:.2f}'.format(f1_score(y_test, y_pred,average='weighted')))
```

```
Accuracy: 0.69

Micro Precision: 0.69
Micro Recall: 0.69
Micro F1-score: 0.69

Macro Precision: 0.64
Macro Recall: 0.65
Macro F1-score: 0.64

Weighted Precision: 0.68
Weighted Recall: 0.69
Weighted F1-score: 0.68
```

TUNNING THE MODEL DECISION TREE

In [51]:
```
from scipy.stats import randint #to Import the randint
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {"max_depth": [3, None], #to Setup the parameters and distributions(maximum and minimum)
              "max_features": randint(1, 13),
              "min_samples_leaf": randint(1, 13),
              "criterion": ["gini", "entropy"]}

#to Initiate the Decision Tree classifier: tree
tree = DecisionTreeClassifier()

#to initiate and Randomized the SearchCV object: tree_cv
tree_cv = RandomizedSearchCV(tree, param_dist, cv=5)

tree_cv.fit(X,y)

print("Tuned Decision Tree Parameters: {}".format(tree_cv.best_params_)) #to Print the tuned decision parameters and score
print("Best score is {}".format(tree_cv.best_score_))
```

```
Tuned Decision Tree Parameters: {'criterion': 'entropy', 'max_depth': None, 'max_features': 4, 'min_samples_leaf': 12}
Best score is 0.929
```

In [52]:
```
from numpy import mean #evaluation of RFE classification
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

#to define the  dataset
X, y = make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

# create pipeline for RFE : Decision tree
rfe = RFE(estimator=DecisionTreeClassifier(), n_features_to_select=13)
model = DecisionTreeClassifier(criterion='gini', max_depth= None, max_features=12, min_samples_leaf=1)
pipeline = Pipeline(steps=[('s',rfe),('m',model)])

# evaluation
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0)
n_scores = cross_val_score(pipeline, X, y, scoring='accuracy', cv=cv,n_jobs=-1, error_score='raise')


print('Accuracy score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print the accuracy score
```

```
Accuracy score: 0.932 (0.020)
```

In [54]:
```
from numpy import mean #evaluation of RFE classification
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

#to define the  dataset
X, y = make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

# create pipeline for RFE : Decision tree
rfe = RFE(estimator=DecisionTreeClassifier(), n_features_to_select=13)
model = DecisionTreeClassifier(criterion='gini', max_depth= None, max_features=12, min_samples_leaf=1)
pipeline = Pipeline(steps=[('s',rfe),('m',model)])

#to evaluate the model
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0)
n_scores = cross_val_score(pipeline, X, y, scoring='precision', cv=cv,n_jobs=-1, error_score='raise')


print('Precision score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print the precision score
```

```
Precision score: 0.936 (0.026)
```

In [55]:
```
from numpy import mean #evaluation of RFE classification
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

#to define the dataset
X, y = make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)


rfe = RFE(estimator=DecisionTreeClassifier(), n_features_to_select=13) # create pipeline for RFE : Decision tree
model = DecisionTreeClassifier(criterion='gini', max_depth= None, max_features=12, min_samples_leaf=1)
pipeline = Pipeline(steps=[('s',rfe),('m',model)])

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0) #evaluation of the model
n_scores = cross_val_score(pipeline, X, y, scoring='recall', cv=cv, n_jobs=-1,error_score='raise')

print('Recall score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print and recall the score
```

```
Recall score: 0.921 (0.040)
```

In [56]:
```
from numpy import mean #evaluation of RFE classification
from numpy import std
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import RFE
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

#to define the dataset
X, y = make_classification(n_samples=1000, n_features=13, n_informative=5,n_redundant=5, random_state=0)

rfe = RFE(estimator=DecisionTreeClassifier(), n_features_to_select=13) # create pipeline for RFE : Decision tree
model = DecisionTreeClassifier(criterion='gini', max_depth= None, max_features=12, min_samples_leaf=1)
pipeline = Pipeline(steps=[('s',rfe),('m',model)])

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=0) #evaluation of the model
n_scores = cross_val_score(pipeline, X, y, scoring='f1', cv=cv, n_jobs=-1,error_score='raise')

print('F1-score: %.3f (%.3f)' % (mean(n_scores), std(n_scores))) #to print and recall the score
```

```
F1-score: 0.931 (0.023)
```